Pretest — Phân tích điểm, xác suất và Gradient Descent

1. Đề bài tiếng Anh

PRETEST

A teacher collects data from five students, including their study hours, attendance rate, and actual exam scores.

Student	Study Hours	Attendance (%)	Actual Score
Student A	2	60	50
Student B	3	65	55
Student C	4	75	65
Student D	5	80	70
Student E	6	95	90

Requirements

Task 1 – Student Performance Analysis and Prediction

1. Create a NumPy matrix student_data containing study hours, attendance, and actual scores from the given dataset.
2. Use the matrix to calculate the mean and standard deviation of actual scores.
3. A simple prediction model is defined as:

   predicted_score = 5 * study_hours + 0.2 * attendance + 10

   Use the matrix to calculate the predicted scores for all five students.

Task 2 – Probability

1. Calculate the probability that a randomly selected student has an actual score greater than or equal to 70.
2. Calculate the probability that a student has an actual score greater than or equal to 70, given that the student studies at least 4 hours.

Task 3 – Gradient Descent

A machine learning model uses the loss function:

[L(w) = (w - 4)^2]

1. Calculate the loss when (w=2).
2. Calculate the gradient when (w=2).
3. Perform one Gradient Descent update using learning rate (α=0.1) and initial value (w=2).
4. Briefly explain why Gradient Descent updates the parameter (w).

2. Đề bài tiếng Việt

BÀI KIỂM TRA ĐẦU VÀO — PRETEST

Một giáo viên thu thập dữ liệu của **năm học sinh**, gồm số giờ học, tỷ lệ tham gia lớp học và điểm thi thực tế.

Học sinh	Số giờ học	Điểm danh (%)	Điểm thi thực tế
Học sinh A	2	60	50
Học sinh B	3	65	55
Học sinh C	4	75	65
Học sinh D	5	80	70
Học sinh E	6	95	90

Yêu cầu

Bài 1 — Phân tích và dự đoán kết quả học tập

1. Tạo một ma trận NumPy (thư viện tính toán mảng) tên là student_data chứa số giờ học, tỷ lệ điểm danh và điểm thi thực tế của năm học sinh theo bảng trên.
2. Sử dụng ma trận để tính **điểm trung bình** và **độ lệch chuẩn** của điểm thi thực tế.
3. Mô hình dự đoán đơn giản đã được định nghĩa:

   Điểm dự đoán = 5 × số giờ học + 0,2 × tỷ lệ điểm danh + 10.

   Sử dụng ma trận để tính điểm dự đoán cho cả năm học sinh.

Bài 2 — Xác suất

1. Tính xác suất một học sinh được chọn ngẫu nhiên có điểm thi thực tế **từ 70 trở lên**.
2. Tính xác suất học sinh có điểm thi thực tế **từ 70 trở lên**, với điều kiện học sinh đó học **ít nhất 4 giờ**.

Bài 3 — Gradient (độ dốc của hàm) Descent (thuật toán giảm độ dốc)

Một mô hình học máy sử dụng hàm mất mát:

[L(w) = (w - 4)^2]

1. Tính giá trị hàm mất mát khi **w = 2**.
2. Tính gradient (đạo hàm theo w) khi **w = 2**.
3. Thực hiện **một lần cập nhật** Gradient Descent với tốc độ học **α = 0,1** và giá trị khởi tạo **w = 2**.
4. Giải thích ngắn gọn tại sao Gradient Descent cập nhật tham số w.

3. Code Python có chú thích

```
import numpy as np

STUDENT_NAMES = ["Student A", "Student B", "Student C", "Student D", "Student E"]

student_data = np.array([
    [2, 60, 50],
    [3, 65, 55],
    [4, 75, 65],
    [5, 80, 70],
    [6, 95, 90],
], dtype=float)


def task1_student_performance(data):  # Phân tích bảng 5 sinh viên và so sánh dự đoán với thực tế.
    if data.shape != (5, 3):
        raise ValueError("Expected the original 5-student x 3-column table.")

    study_hours = data[:, 0]
    attendance = data[:, 1]
    actual_scores = data[:, 2]

    mean_score = float(np.mean(actual_scores))  # Tính trung bình cộng các giá trị hợp lệ.

    population_sd = float(np.std(actual_scores, ddof=0))  # Tính độ lệch chuẩn để đo sự phân tán.
    sample_sd = float(np.std(actual_scores, ddof=1))  # Tính độ lệch chuẩn để đo sự phân tán.

    predicted_scores = 5 * study_hours + 0.2 * attendance + 10
    return mean_score, population_sd, sample_sd, predicted_scores


def task2_probabilities(data):  # Đếm số điều kiện đúng bằng Boolean và tính xác suất.
    scores = data[:, 2]
    hours = data[:, 0]

    at_least_70 = scores >= 70
    at_least_4_hours = hours >= 4
    both_conditions = at_least_70 & at_least_4_hours

    probability_70 = float(np.count_nonzero(at_least_70) / len(scores))  # Đếm hoặc cộng phần tử để tính tổng và xác suất.

    denominator = int(np.count_nonzero(at_least_4_hours))  # Đếm hoặc cộng phần tử để tính tổng và xác suất.
    if denominator == 0:
        raise ValueError("No students meet the given study-hours condition.")
    conditional_probability = float(
        np.count_nonzero(both_conditions) / denominator  # Đếm hoặc cộng phần tử để tính tổng và xác suất.
    )
    return probability_70, conditional_probability


def task3_gradient_descent(initial_w=2.0, learning_rate=0.1):  # Thực hiện đúng một bước cập nhật tham số w.
    loss_before = (initial_w - 4) ** 2

    gradient = 2 * (initial_w - 4)
```

```
    updated_w = initial_w - learning_rate * gradient

    loss_after = (updated_w - 4) ** 2
    return loss_before, gradient, updated_w, loss_after


def main():
    mean, pop_sd, sample_sd, predicted = task1_student_performance(student_data)
    print("TASK 1 — STUDENT PERFORMANCE")
    print("Actual-score mean:", f"{mean:.2f}")
    print("Population SD (NumPy default, ddof=0):", f"{pop_sd:.6f}")
    print("Sample SD (comparison only, ddof=1):", f"{sample_sd:.6f}")
    print("Student | actual score | predicted score")
    for name, actual, prediction in zip(
        STUDENT_NAMES, student_data[:, 2], predicted
    ):
        print(f"{name} | {actual:.0f} | {prediction:.0f}")

    p70, p70_given_4h = task2_probabilities(student_data)
    print("TASK 2 — PROBABILITY")
    print("P(actual score >= 70):", f"{p70:.6f}")
    print("P(actual score >= 70 | hours >= 4):", f"{p70_given_4h:.6f}")

    loss, derivative, new_w, new_loss = task3_gradient_descent()
    print("TASK 3 — GRADIENT DESCENT")
    print("L(2):", f"{loss:.2f}")
    print("Gradient at w=2:", f"{derivative:.2f}")
    print("Updated w after one step:", f"{new_w:.2f}")
    print("Loss after one update (extra check):", f"{new_loss:.2f}")

    if not (student_data.shape == (5, 3)): raise ValueError("Kiểm tra không đạt")
    if not (np.isclose(mean, 66.0)): raise ValueError("Kiểm tra không đạt")
    if not (np.isclose(pop_sd, np.sqrt(194.0))): raise ValueError("Kiểm tra không đạt")
    if not (np.allclose(predicted, [32, 38, 45, 51, 59])): raise ValueError("Kiểm tra không đạt")
    if not (np.isclose(p70, 2 / 5)): raise ValueError("Kiểm tra không đạt")
    if not (np.isclose(p70_given_4h, 2 / 3)): raise ValueError("Kiểm tra không đạt")
    if not (np.isclose(loss, 4.0)): raise ValueError("Kiểm tra không đạt")
    if not (np.isclose(derivative, -4.0)): raise ValueError("Kiểm tra không đạt")
    if not (np.isclose(new_w, 2.4)): raise ValueError("Kiểm tra không đạt")
    if not (new_loss < loss): raise ValueError("Kiểm tra không đạt")


if __name__ == "__main__":
    main()
```

4. Giải thích nội dung và cách giải

Phần 3 chứa **một chương trình Python thống nhất**. Dưới đây là giải thích chi tiết cách từng nhóm hàm giải quyết yêu cầu, không lặp lại các đoạn code rời.

Câu 1 — Phân tích điểm và dự đoán

Mục tiêu: Dùng bảng của năm học sinh để tính trung bình, độ lệch chuẩn của **điểm thực tế**, sau đó dự đoán điểm bằng biểu thức được đề cung cấp. Không tự huấn luyện mô hình.

Vì sao làm theo thứ tự này? Cần chuyển bảng thành mảng số trước, lấy đúng cột điểm thi để thống kê, rồi lấy cột giờ học và điểm danh để dự đoán. Nếu trộn các cột, kết quả sẽ sai dù cú pháp Python đúng.

Quy trình giải:

1. Tạo mảng có năm hàng, ba cột. Mỗi hàng là một học sinh; các cột lần lượt là giờ học, tỷ lệ điểm danh và điểm thi. `dtype=float` yêu cầu lưu dạng số thực để thuận tiện tính toán.
2. Lấy cột điểm thực tế bằng `data[:, 2]`. Dấu `:` chọn tất cả hàng, số `2` chọn cột thứ ba vì Python đếm cột từ 0.
3. Dùng `np.mean(scores)` tính trung bình. Dùng `np.std(scores, ddof=0)` tính độ lệch chuẩn của toàn bộ nhóm năm học sinh. Đề không nêu mẫu hay tổng thể, nên cũng tính thêm `ddof=1` để người học so sánh trường hợp xem năm học sinh là mẫu.
4. Tính đồng thời điểm dự đoán theo hệ số đề cho; không đổi 60% thành 0.6, vì biểu thức của đề sử dụng trực tiếp giá trị điểm danh 60, 65, ...

Công thức viết bằng Python:

Giải thích hàm, toán tử và tham số: `np.array` tạo mảng, `dtype=float` chọn kiểu số thực. `np.mean` lấy trung bình. `np.std` đo độ phân tán; `ddof=0` chia phương sai cho số quan sát `n`, còn `ddof=1` chia cho `n-1`. Phép `*` nhân từng phần tử trong mảng NumPy, `+` cộng. `data[:, 0]` lấy giờ học, `data[:, 1]` lấy điểm danh, `data[:, 2]` lấy điểm thực tế.

Ví dụ và kết quả đối chiếu: Trung bình **66**, độ lệch chuẩn tổng thể khoảng **13.9284**, độ lệch chuẩn mẫu khoảng **15.5724**. Điểm dự đoán lần lượt **32, 38, 45, 51, 59**; học sinh A có `5 * 2 + 0.2 * 60 + 10` bằng **32**. Không nhầm điểm dự đoán với điểm thực tế.

Từ vựng: *Mean* (trung bình), *standard deviation* (độ lệch chuẩn), *prediction* (dự đoán).

Câu 2 — Xác suất và xác suất có điều kiện

Mục tiêu: Tìm xác suất đạt ít nhất 70 điểm khi chọn bất kỳ học sinh nào; sau đó tìm xác suất đạt ít nhất 70 điểm **nếu đã biết** học sinh đó học ít nhất 4 giờ.

Tại sao phải giải khác nhau? Câu đầu xét đủ **5 học sinh**, trong đó D và E đạt 70 điểm trở lên. Câu sau chỉ xét nhóm **C, D, E**, gồm **3 học sinh** học từ 4 giờ; trong nhóm đó D và E đạt ít nhất 70 điểm. Xác suất có điều kiện phải dùng đúng số người trong nhóm được biết trước làm mẫu số.

Quy trình giải:

1. So sánh mảng điểm với 70 để tạo các giá trị đúng/sai; dùng `np.count_nonzero` đếm số học sinh đạt.
2. Lấy số đạt (**2**) chia cho tổng số (**5**).
3. So sánh giờ học với 4 để chọn nhóm điều kiện; lấy `&` kết hợp hai điều kiện để đếm người vừa học đủ giờ vừa đạt điểm.
4. Lấy số người thỏa cả hai (**2**) chia cho số người học từ 4 giờ (**3**). Phải kiểm tra mẫu số khác 0.

Công thức viết bằng Python:

Giải thích hàm, toán tử và flag: `>=` là lớn hơn **hoặc bằng**, nên điểm đúng 70 vẫn được tính. `&` có nghĩa là *AND* (và) theo từng vị trí của **mảng Boolean** (mảng đúng/sai). `np.count_nonzero` đếm số phần tử đúng. `len` đếm tổng số phần tử, `/` là phép chia thực. Trong f-string (chuỗi định dạng) `:.0%` hiển thị phần trăm không có chữ số thập phân, `:.2%` hiển thị hai chữ số thập phân; đây là **định dạng in**, không sửa giá trị xác suất. `if count_4h == 0` chặn trường hợp chia cho 0.

Ví dụ và kết quả đối chiếu: `p_pass` bằng **0.4 (40%)**; `p_conditional` khoảng **0.6667 (66.67%)**. Không dùng lại mẫu số 5 cho xác suất có điều kiện.

Từ vựng: *Probability* (xác suất), *conditional probability* (xác suất có điều kiện), *Boolean* (đúng/sai).

Câu 3 — Một bước Gradient Descent

Mục tiêu: Tính *loss* (mức mất mát) tại `w = 2`, tính *gradient* (độ dốc/đạo hàm), cập nhật tham số `w` **một lần** với tốc độ học `0.1` và giải thích tại sao cần cập nhật.

Vì sao dùng gradient? Loss cho biết tham số hiện tại chưa phù hợp đến đâu. Gradient cho biết hướng tăng của loss; muốn giảm loss, ta điều chỉnh tham số theo **hướng ngược gradient**. Với `w = 2`, gradient âm nghĩa là cần tăng `w` để tiến gần giá trị tối ưu `4`.

Quy trình giải:

1. Viết biểu thức loss của đề bằng `(w - 4) ** 2`. Với `w=2`, giá trị là **4**.
2. Áp dụng quy tắc đạo hàm của bình phương, viết `gradient = 2 * (w - 4)`. Với `w=2`, gradient là **-4**.
3. Tính `updated_w = w - learning_rate * gradient`. Vì gradient âm, kết quả mới là **2.4**, tức `w` tăng lên.
4. Dùng biểu thức loss ban đầu để kiểm tra tại `updated_w`. Loss giảm còn **2.56**; không cần lặp nhiều lần vì đề yêu cầu đúng một bước.

Công thức viết bằng Python:

Giải thích toán tử, biến và tham số: `** 2` là lũy thừa bậc hai trong Python, **không dùng ^** (toán tử XOR). `*` là nhân và `-` là trừ. `learning_rate=0.1` là kích thước bước cập nhật: quá lớn có thể vượt qua điểm thấp nhất; quá nhỏ làm tiến triển chậm. `gradient` là đạo hàm theo `w`, không phải xác suất. `loss_after` chỉ là phép kiểm tra thêm; bài không yêu cầu `epochs` (số vòng học).

Ví dụ và kết quả đối chiếu: Loss ban đầu **4**, gradient **-4**, giá trị mới **2.4**, loss sau cập nhật **2.56**. Không trộn số liệu học sinh ở Câu 1 vào hàm loss riêng của Câu 3.

Từ vựng: *Gradient Descent* (thuật toán giảm độ dốc), *loss* (hàm mất mát), *learning rate* (tốc độ học), *parameter* (tham số).

Ba câu hỏi liên kết kiến thức như thế nào?

Câu 1 — Từ bảng dữ liệu đến thống kê: Ma trận `student_data` có **5 hàng × 3 cột**; mỗi hàng là một học viên, lần lượt gồm giờ học, điểm thực tế và điểm dự đoán. Cú pháp `data[:, 1]` lấy tất cả điểm thực tế, `data[:, 2]` lấy tất cả dự đoán. Trung bình điểm thực tế dùng `np.mean`; độ lệch chuẩn quần thể dùng `np.std(..., ddof=0)` và độ lệch chuẩn mẫu dùng `ddof=1`. Nếu muốn biết mô hình dự đoán sai bao nhiêu, hãy đối chiếu từng cặp điểm thật/dự đoán, không so toàn bộ trung bình rồi coi đó là sai số từng người.

Câu 2 — Boolean và xác suất: `scores >= 70` không cho ra một số 0/1 duy nhất, mà cho **mảng Boolean** cùng kích thước với `scores`. Mỗi phần tử đúng là `True`, sai là `False`. `np.count_nonzero(mask)` trả **số lượng phần tử đúng**; `np.mean(mask)` sẽ trả tỷ lệ đúng vì trong phép cộng, True tương đương 1, False tương đương 0. Xác suất `P(điểm ≥70)` bằng số người đạt điều kiện chia 5, không phải kết quả Boolean. Xác suất có điều kiện `P(A|B)` phải chia **số người thuộc B**, không chia tất cả 5 người; phải xử lý trường hợp nhóm B rỗng trước khi chia.

Câu 3 — Gradient Descent: Với `L(w)=(w-4)²`, đạo hàm là `2(w-4)`. Tại `w=2`, gradient bằng `-4`; dùng `learning_rate=0.1` thì `w_mới=2-0.1×(-4)=2.4`. Loss trước bằng 4 và sau bằng 2.56, nên bước này đi đúng chiều giảm loss. Đây là **một bước cập nhật**, không phải huấn luyện tới hội tụ.

Khi ôn thi: phân biệt độ lệch chuẩn với sai số chuẩn, số đếm với xác suất, `and` điều kiện với `&` trên mảng Boolean NumPy, và dấu của gradient khi trừ một số âm. Hầu hết lỗi sai đến từ chọn sai mẫu số hoặc đảo dấu cập nhật chứ không phải lệnh thư viện.